Workgroup:	Network Working Group
Internet-Draft:	draft-wullink-rpp-oauth2-00
Published:	6 July 2026
Intended Status:	Standards Track
Expires:	7 January 2027
Authors:	M. Wullink *SIDN Labs*	P. Kowalik *DENIC*
---	---	---

OAuth 2.0 for RESTful Provisioning Protocol (RPP)

Abstract

This document describes how OAuth 2.0 [RFC6749] can be used to secure RESTful Provisioning Protocol (RPP) API requests described in [I-D.ietf-rpp-core].

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at https://datatracker.ietf.org/drafts/current/.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 7 January 2027.

Copyright Notice

Table of Contents

1. Introduction

In order to allow for fine-grained access control, which is a key design goal of RPP's authorization model, a registrar can operate multiple user accounts within the registry. Each account carries a distinct set of permissions appropriate to the user's or tool's role (e.g., read-only reporting accounts, accounts limited to a specific set of operations, or fully privileged administrative accounts). This allows registrars to implement the principle of least privilege within their own organizations without requiring separate registry-level registrar accounts.

Due to the stateless nature of RPP, the client includes authorization credentials in each HTTP request. RPP uses OAuth 2.0 [RFC6749] for delegated authorization via Bearer tokens. Basic authentication [RFC7617] SHOULD NOT be used. The server MUST validate the Bearer token on each request and reject any request with an invalid or expired token with an appropriate HTTP status code.

2. Terminology

In this document the following terminology is used.

URL - A Uniform Resource Locator as defined in [RFC3986].

Resource - An object having a type, data, and possible relationship to other resources, identified by a URL.

RPP client - An HTTP user agent performing an RPP request

RPP server - An HTTP server responsible for processing requests and returning results in any supported media type.

JWT - JSON Web Token as defined in [RFC7519].

Authorization Server (AS) - A server that issues OAuth 2.0 access tokens to clients after successfully authenticating the resource owner and obtaining authorization, as defined in [RFC6749].

Resource Server (RS) - A server hosting protected resources that accepts and validates OAuth 2.0 access tokens to authorize requests, as defined in [RFC6749].

Client - An application making protected resource requests on behalf of the resource owner and with its authorization, as defined in [RFC6749].

Resource Owner - An entity capable of granting access to a protected resource. When the resource owner is a person, it is referred to as an end-user, as defined in [RFC6749].

Access Token - A credential used by a client to access protected resources. In RPP, access tokens MUST be JWTs conforming to [RFC9068].

Bearer Token - A type of access token where any party in possession of the token can use it to access the associated resource, as defined in [RFC6750].

Client Credentials Grant - An OAuth 2.0 grant type in which the client authenticates directly with the AS using its own credentials to obtain an access token, without end-user involvement, as defined in Section 4.4. Used for machine-to-machine flows.

Authorization Code Grant - An OAuth 2.0 grant type in which the client obtains an authorization code from the AS via a user-agent redirect, then exchanges it for an access token, as defined in Section 4.1. Used for interactive flows involving end-users.

PKCE (Proof Key for Code Exchange) - An extension to the Authorization Code Grant that prevents authorization code interception attacks, as defined in [RFC7636].

Scope - A mechanism in OAuth 2.0 to limit the access granted by an access token, as defined in [RFC6749]. RPP uses scopes to enforce fine-grained access control over provisioning operations.

OAuth 2.0 AS Metadata - A mechanism for ASs to publish their configuration and capabilities at a well-known URL, as defined in [RFC8414].

Rich Authorization Requests (RAR) - An OAuth 2.0 extension that allows clients to request fine-grained authorization data beyond what scopes can express, as defined in [RFC9396].

3. Conventions Used in This Document

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT","SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in [RFC2119].

In examples, indentation and white space are provided only to illustrate element relationships and are not REQUIRED features of the protocol.

All example requests assume an RPP server using HTTP version 2 is listening on the standard HTTPS port on host rpp.example. An authorization token has been provided by an out-of-band process and MUST be used by the client to authenticate each request.

4. Architectural Overview

The diagram below gives an overview of all actors and their relationships in the RPP OAuth 2.0 architecture. The Registry operates both the Authorization Server (AS) and the RPP server. The Registrar operates the Registrar Backend, which is an RPP client, and may also operate its own AS. The Registry Employee and Registrar Employee are human operators that interact with the registry and registrar systems via a web browser. The Registry Client App is a client application operated by the registry on behalf of the Registry Employee, while the Registrar Backend is a client application operated by the registrar on behalf of the Registrar Employee and for automated operations. Both client applications interact with the RPP server using OAuth 2.0 Bearer tokens for authorization.

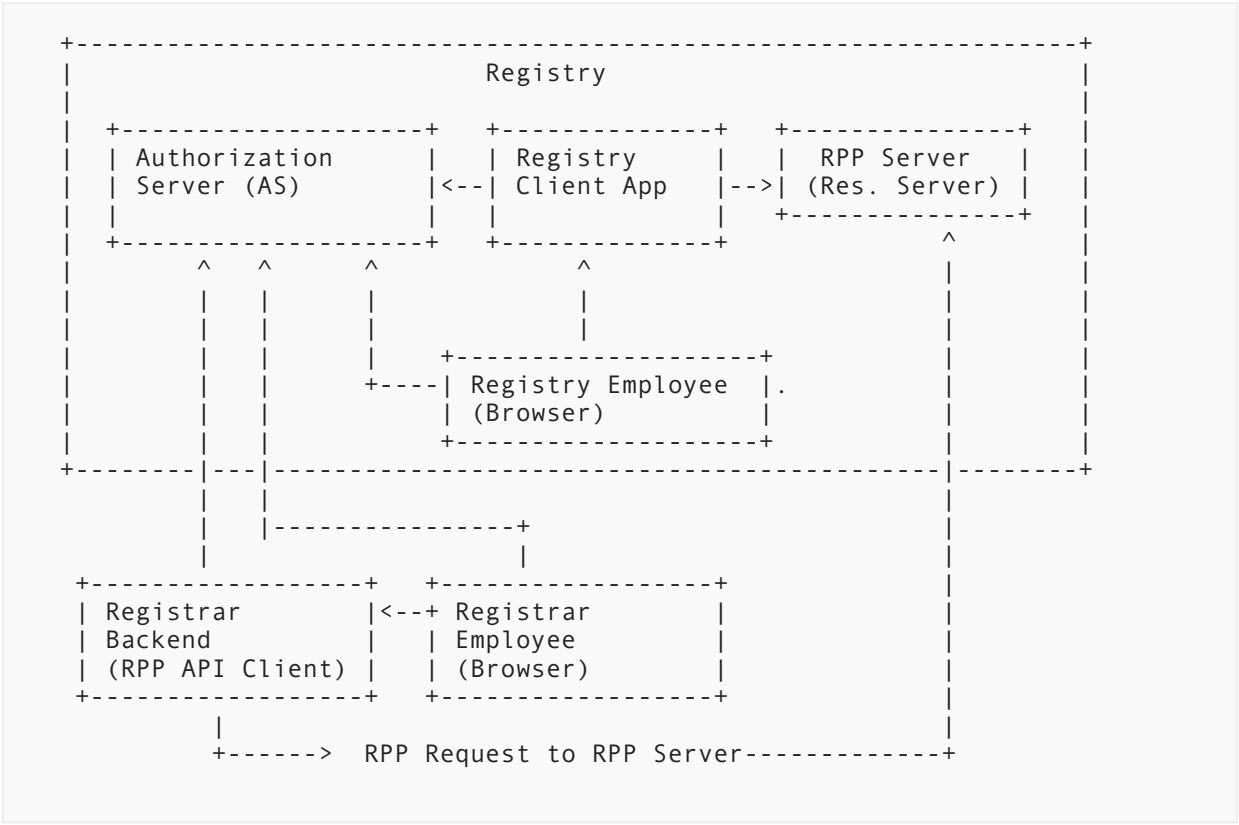

Figure 1: RPP OAuth 2.0 Architecture Overview

The actors in the diagram are as follows:

- **Registry AS**: The OAuth 2.0 Authorization Server operated by the registry. All access tokens are issued by this AS. It is the single trust anchor for RPP authorization.
- **Registry RPP Server**: The OAuth 2.0 Resource Server that exposes the RPP API. It validates Bearer tokens issued by the Registry AS before processing any request.
- **Registry Client App**: A client application operated by the registry. The Registry Employee authenticates with the AS via the Authorization Code grant, receives the token, and uses the Registry Client App. The Registry Client App also exchanges the authorization code for a token directly at the AS, then sends RPP requests to the RPP server on the employee's behalf.
- **Registry Employee (Browser)**: A human operator at the registry. Authenticates directly with the Registry AS using the Authorization Code grant, receives a token, and uses the Registry Client App to interact with the RPP server.
- **Registrar Backend (RPP API Client)**: The registrar's backend system that communicates directly with the RPP server. It obtains tokens using either the Client Credentials grant for automated M2M operations, or the Authorization Code grant when acting on behalf of an authenticated registrar employee. It sends all RPP requests to the RPP server.
- **Registrar Employee (Browser)**: A human operator at the registrar. Authenticates directly with the Registry AS using the Authorization Code grant, after which the token is delivered to the Registrar Backend via redirect callback. The employee uses the Registrar Backend to interact with the RPP server.

5. Authorization

RPP MAY use OAuth 2.0 [RFC6749] as its authorization framework. OAuth 2.0 is an authorization protocol and does not perform authentication itself; authentication of the client or end-user is handled by the AS before a token is issued. RPP acts as an OAuth 2.0 Resource Server and MUST validate every incoming request against a Bearer token presented in the `Authorization` header. The registry MAY operate its own Authorization Server (AS) or MAY delegate to an external AS. Access control decisions are derived exclusively from claims in the presented JWT access token.

Access tokens MUST be JWTs conforming to the JWT Profile for OAuth 2.0 Access Tokens [RFC9068]. Tokens MUST be signed using asymmetric cryptography; symmetric signing algorithms (e.g., HS256) MUST NOT be used for tokens issued by external ASs. Short-lived tokens are RECOMMENDED and token caching and refresh strategies MUST follow the best practices defined in [RFC8725].

An RPP server determines what AS issued a token by inspecting the `iss` claim; the RPP server MUST validate the token's signature against the issuing AS's public key, which may be fetched and cached via OAuth 2.0 AS Metadata [RFC8414] or by an out-of-band mechanism.

In both modes authorization is enforced identically. The `aud` claim MUST identify the RPP server as the intended audience; the RPP server MUST reject tokens where its own identifier is absent from `aud`.

RPP MUST support the `Client Credentials Grant` grant type described in Section 4.4 to allow registrar client systems to obtain access tokens.

6. Scopes

OAuth 2.0 scopes are used for granting authorization and enforcing access control when accessing RPP resources. The server MUST define a set of scopes that can be requested by clients when obtaining access tokens. The server MUST also define the mapping between scopes and the specific resources and operations that they grant access to.

RPP scopes are based on the objects, processes and operations defined in [I-D.ietf-rpp-data-objects]. Each scope corresponds to a specific set of permissions for accessing and manipulating RPP resources.

6.1. Scope Derivation Rules

RPP scopes are derived systematically from the data object types and operation categories defined in [I-D.ietf-rpp-data-objects]. The derivation rules are as follows:

- The scope identifier MUST use the format `<object>:<access-level>`, where `<object>` is the lowercase stable identifier of the data object and `<access-level>` is one of the access levels defined below based on the object operation.
- The `create` access level grants permission to perform the Create operation.
- The `read` access level grants permission to perform the Read operation.
- The `update` access level grants permission to perform the Update operation.
- The `renew` access level grants permission to perform the Renew operation.
- The `restore` access level grants permission to perform the Restore operation.
- The `delete` access level grants permission to perform the Delete operation.
- The `transfer` access level grants permission to perform all Transfer operations.

- The `list` access level grants permission to perform List operations.

6.2. Scope Registry

Table Table 1 defines the RPP scopes derived from the data objects specified in [I-D.ietf-rpp-data-objects].

Scope	Data Object	Operations Granted
`domain:create`	Domain Name	Create
`domain:read`	Domain Name	Read
`domain:update`	Domain Name	Update
`domain:renew`	Domain Name	Renew
`domain:restore`	Domain Name	Restore
`domain:delete`	Domain Name	Delete
`domain:transfer`	Domain Name	Create, Approve, Reject, Cancel, Query
`domain:list`	Domain Name	List domain collection
`contact:create`	Contact	Create
`contact:read`	Contact	Read
`contact:update`	Contact	Update, Restore
`contact:delete`	Contact	Delete
`contact:transfer`	Contact	Create, Approve, Reject, Cancel, Query
`contact:list`	Contact	List contact collection
`host:create`	Host	Create
`host:read`	Host	Read
`host:update`	Host	Update, Restore
`host:delete`	Host	Delete
`host:list`	Host	List host collection

Table 1: RPP OAuth 2.0 Scopes

TODO: add more scopes, such as for listing collections, process statuses, and administrative scopes.

7. Object-Specific Authorization

An RPP process may require conveying the specific object being processed in the authorization request so that the registrant can give informed consent for that specific object. This is necessary to prevent overbroad consent where a registrant might

unknowingly authorize unwanted operations on their objects. Conveying the specific object also allows the AS to enforce fine-grained access control and ensures that the registry has verifiable evidence of exactly which object was authorized.

OAuth 2.0 Rich Authorization Requests (RAR) [RFC9396] extends the standard OAuth 2.0 authorization request with an `authorization_details` parameter that carries a structured JSON object describing precisely what the client is requesting authorization for. Unlike scopes, which are coarse-grained string tokens, `authorization_details` allows the request to include typed, fine-grained authorization data, such as the specific object being acted upon. The AS can present this information to the user in a meaningful consent screen.

Table Table 2 lists the RAR fields defined for RPP.

Field	Type	Requirement	Description
`type`	String	REQUIRED	The type of RPP operation being authorized.
`object_type`	String	REQUIRED	The type of RPP object the operation is being performed upon.
`object_identifier`	String	REQUIRED	The unique identifier of the specific object the operation applies to (e.g., `foo.example` or `CID-12345`).

Table 2: RPP Transfer Authorization, RAR `authorization_details` *object (Primary Method, [RFC9396])*

Example RAR `authorization_details` value for a domain transfer:

```
[{
  "type": "transfer",
  "object_type": "domain",
  "object_identifier": "foo.example"
}]
```

The AS MUST echo the `authorization_details` back as a claim in the issued JWT. The registry MUST validate the `authorization_details` claim in the token and MUST verify that `object_type` and `object_identifier` match the object to which the operation is being applied.

8. Claims

The JWT Profile for OAuth 2.0 Access Tokens defined in [RFC9068] defines a standard set of claims that MUST be present in every access token, such as `iss`, `sub`, `aud`, `exp`, and `scope`. RPP also defines additional custom claims that are specific to RPP, such as `rpp_registrar_id` (see Section 8.2). These claims provide the necessary information for the RPP server to make informed access control decisions based on the identity of the requester, the registrar they represent, and the specific permissions granted by their token.

8.1. JWT Profile Claims

The JWT Profile for OAuth 2.0 Access Tokens defined in [RFC9068] specifies a standard set of claims that MUST be included in every access token issued by the AS.

Table Table 3 lists the JWT Profile claims that MUST be present in every RPP access token:

Claim	Type	Description
`iss`	String (URI)	Identifies the issuing AS. The RPP server MUST validate this against its set of trusted issuers.
`sub`	String	The subject of the token. For machine-to-machine flows this MUST be the client identifier. For interactive flows this MUST be the end-user identifier of the authenticated end-user at the issuing AS. The end-user MAY be a registrar employee operating the registrar's management system, or a registry employee using the registry client application.
`aud`	String or Array	Identifies the intended audience. MUST include the RPP server's resource identifier. The RPP server MUST reject tokens where its own identifier is not present in this claim.
`exp`	Numeric date	Expiry time. The RPP server MUST reject tokens that have expired.
`iat`	Numeric date	Time at which the token was issued.
`jti`	String	Unique identifier for the token, used to prevent token replay attacks.
`client_id`	String	The OAuth 2.0 client identifier of the RPP client application.
`scope`	String	Space-separated list of granted scopes (see Section 6). The RPP server MUST enforce access control based on the scopes present in this claim.

Table 3: OAuth 2.0 Access Token Claims for RPP ([RFC9068])

8.2. RPP-Specific Claims

In addition to the standard JWT Profile claims defined in [RFC9068], table Table 4 lists the RPP-specific claims that are defined to enable fine-grained authorization decisions. Required claims MUST be present in every RPP access token. Optional claims SHOULD be included when applicable to the deployment or request context.

Claim	Requirement	Type	Description
`rpp_registrar_id`	REQUIRED	String	The identifier of the registrar on whose behalf the request is made. The RPP server MUST validate that this identifier matches a known and authorized registrar. This claim MUST be present in all access tokens used for RPP requests.

Claim	Requirement	Type	Description
`rpp_reseller_id`	OPTIONAL	String	The identifier of the reseller acting through the registrar's client application. This claim SHOULD be included when the request originates from a reseller operating under the registrar's account. The RPP server MAY use this claim for access control, auditing, and attribution purposes. The value is interpreted within the namespace of the registrar identified by `rpp_registrar_id`.

Table 4: RPP Specific Access Token Claims

The combination of the `sub` claim and the `rpp_registrar_id` claim provides a complete, two-dimensional identity for every RPP request: `sub` identifies the individual principal (registrar employee, automated process, or registrar customer) that initiated the request, while `rpp_registrar_id` identifies the registrar organization as a whole within the registry's domain.

The identity of the `sub` depends on both the flow type and the identity domain of the principal:

- In **machine-to-machine** flows, `sub` is the registrar's OAuth 2.0 client identifier, representing an automated system acting on behalf of the registrar. The token is issued by the **registry's AS**, which maintains the registrar's client credentials.
- In **interactive flows initiated by registrar staff**, `sub` is the identifier of the registrar employee. Registrar employees are managed as users in the **registry's AS**. The token is therefore also issued by the registry's AS, and the `sub` value is the employee's account identifier within that server. The `iss` claim will identify the registry's AS.
- In **interactive flows initiated by a registrant customer**, the situation is different. Registrant customers are not managed in the registry's AS, they are maintained in the **registrar's own AS** (the registrar acts as the AS for its customers). The token is therefore issued by the registrar's AS, and the `iss` claim will identify the registrar's AS as the issuer. The registry MUST have a pre-established trust relationship with the registrar's AS to accept and validate such tokens. In this case, the `sub` value MUST be the registrant's identifier as it exists in the registry database. The registrar MUST use this registry-assigned id, not any registrar-internal customer identifier, as the `sub` value. This ensures the registry can unambiguously correlate the token's subject to an existing provisioned contact object. This enables verification of ownership and consent for operations. The registry MUST reject tokens where the `sub` value does not match a known contact handle associated with the object being acted upon.

Extensions and profiles MAY define additional claims. All additional claims MUST use a URI or a collision-resistant name as the claim name to prevent conflicts with registered claims.

8.3. Claim Validation

The RPP server MUST validate all required claims in accordance with [RFC9068] and [RFC8725].

9. Data Objects

The RPP Data Object Catalog described in [I-D.ietf-rpp-data-objects] is extended to include new objects required for using OAuth 2.0 as a framework for authorization in RPP.

- *Client Object*: A registrar MUST register at least one OAuth 2.0 client to interact with the RPP server. The Client Object MUST include the following attributes:
  - Name: Unique (in registrar namespace) name of application.
  - Description: A brief description of the application's purpose.
  - Redirect URL: The URL to which the authorization server will redirect the user after granting or denying access.
  - Client Id: The OAuth 2.0 client identifier issued to the client during the registration process.
  - Client Secret: The OAuth 2.0 client secret issued to the client during the registration process.
- TODO

10. Federation

For more advanced use cases, enabled by OAuth 2.0, such as an interactive federated object transfer, it is necessary for the RPP server to validate tokens issued by external ASs operated by registrars. This requires the RPP server to establish trust with these external ASs. When JWTs are used for Client Authentication as specified in [RFC7523], the registrar MUST be able to manage their public key(s) in the registry database.

11. Flows

RPP defines two distinct authorization flows: machine-to-machine flows and interactive flows. Machine-to-machine flows are designed for use in automated systems where no user interaction is required. Interactive flows are designed for use in scenarios where end-user interaction is required, such as when a registrant employee needs to interact with the registry system. For these interactive flows, the OAuth 2.0 Authorization Code grant Section 4.1 MUST be used.

11.1. Machine to Machine

The machine-to-machine (M2M) flow is used for automated RPP requests such as domain provisioning, renewal, or host management sent by a registrar's backend systems to the registry. No end-user interaction is involved. JWTs for Client Authentication as specified in [RFC7523] or the OAuth 2.0 Client Credentials grant Section 4.4 MUST be used.

In this flow the registrar's system authenticates directly with the registry's AS using a signed JWT or a pre-registered `client_id` and `client_secret`. The AS issues a short-lived access token scoped to the requested RPP operations. The registrar's system then includes this token in the `Authorization` header of each RPP request sent to the registry.

The `sub` claim in the resulting token MUST be set to the `client_id` of the registrar's system. The `rpp_registrar_id` claim MUST also be present, identifying the registrar organization within the registry's namespace.

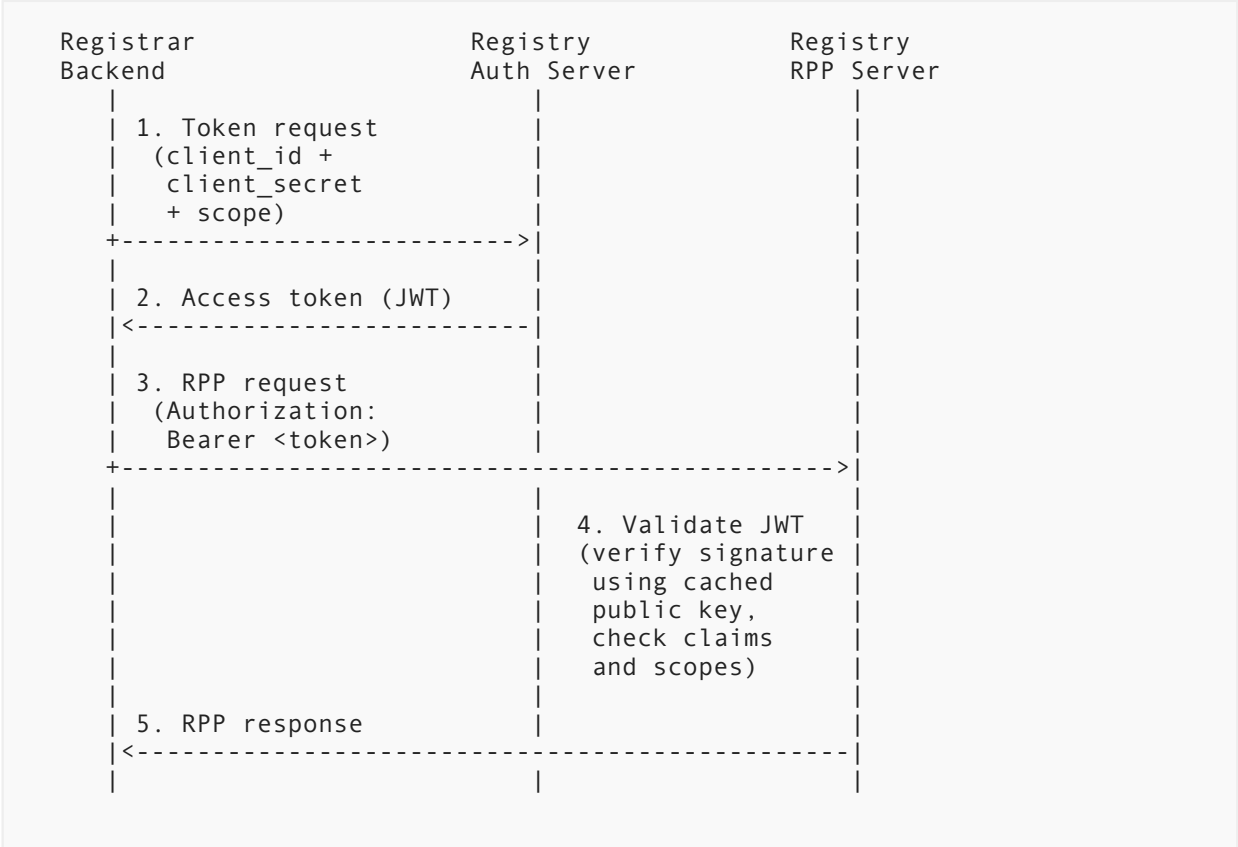

Figure 2: OAuth 2.0 Client Credentials Flow for Registrar-to-Registry Requests

The steps in the diagram are as follows:

1. The registrar's backend system sends a token request to the registry's AS, it SHOULD use JWTs for Client Authentication as specified in [RFC7523] or if this is not supported by the AS, it SHOULD use the Client Credentials Grant, and the requested RPP scopes (e.g., `domain:create`).
2. The AS validates the client credentials and issues a signed, short-lived JWT access token containing the granted scopes, `sub` (set to `client_id`), and `rpp_registrar_id`.
3. The registrar's system sends the RPP request to the registry's RPP server, including the access token in the HTTP `Authorization` header as a Bearer token.
4. The RPP server validates the JWT entirely locally without contacting the AS. It verifies the token's signature using the AS's public key, checks the standard claims (`iss`, `aud`, `exp`), and confirms that the `scope` claim includes the scope required for the requested operation.
5. If validation succeeds, the RPP server processes the request and returns the RPP response.

Example request using JWT Client Authentication ([RFC7523]), using the `domain:create` scope. The client authenticates by presenting a signed JWT assertion instead of a client secret:

```
POST /token HTTP/2
Host: as.rpp.example
Content-Type: application/x-www-form-urlencoded

&scope=domain%3Acreate
&client_assertion_type=urn%3Aietf%3Aparams%3Aoauth%3Aclient-assertion-
type%3Ajwt-bearer
&client_assertion=eyJhbGciOiJSUzI1NiI[...omitted for brevity...]
```

The `client_assertion` is a JWT signed with the registrar's private key. Its payload MUST contain:

- `iss`: the registrar's `client_id`
- `sub`: the registrar's `client_id`
- `aud`: the registry AS token endpoint URI
- `jti`: a unique identifier for this assertion (to prevent replay)
- `exp`: expiry time (SHOULD be short-lived, e.g., 60 seconds)

Example `client_assertion` payload:

```
{
  "iss": "registrar-client-id",
  "sub": "registrar-client-id",
  "aud": "https://authorization-server.rpp.example/token",
  "jti": "unique-jwt-id-123",
  "exp": 1746134400
}
```

Example request using the OAuth 2.0 Client Credentials grant with a `client_secret` (fallback when JWT Client Authentication is not supported):

```
POST /token HTTP/2
Host: authorization-server.rpp.example
Authorization: Basic cmVnaXN0cmFyLWNsaWVudC1pZDpjbGllbnQtc2VjcmV0
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials&scope=domain%3Acreate
```

Response:

```
HTTP/2 200 OK
Content-Type: application/json;charset=UTF-8
Cache-Control: no-store
Pragma: no-cache

{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "scope": "domain:create"
}
```

11.1.1. High-Risk Operations

The machine-to-machine (M2M) Client Credentials flow is appropriate for routine automated provisioning, but certain high-risk operations, e.g. object transfer or modification of authorisation information, MUST require proof that an authorized human principal explicitly approved the action. Without enforcement, a registrar could use M2M tokens for all operations, eliminating individual accountability. The operations for which interactive authentication is required are a matter of registry policy.

The following mechanisms MAY be used by the registry to enforce interactive authentication for designated operations:

`sub` **MUST identify a human principal**: The registry MAY require that for high-risk operations, the `sub` claim MUST contain the identifier of an authenticated human principal and MUST NOT equal the `client_id`. In M2M tokens issued via the Client Credentials grant, `sub` is always set to `client_id`, representing an automated system rather than a human. By mandating `sub` != `client_id` for designated operations, the registry ensures those operations can only be performed with a token issued on behalf of a real, identified individual. The RPP server MUST reject requests for these operations when `sub` equals `client_id`.

Scope restriction by grant type: The registry's AS SHOULD be configured to refuse issuing certain high-risk scopes to the Client Credentials grant type. Only the Authorization Code grant (interactive) MAY be permitted to obtain these scopes. This prevents a registrar from obtaining the necessary scope for a high-risk operation through unattended M2M authentication.

11.2. Interactive

The interactive flow is used for RPP requests that require end-user interaction. The OAuth 2.0 Authorization Code grant Section 4.1 MUST be used for this flow. **Registrar employees** are managed as users in the **registry's AS**. A registrar employee uses the registrar's client application to perform operations on behalf of the registrar, such as updating domain records or managing contacts. The registry's AS authenticates the employee and issues an access token. The `sub` claim will contain the employee's account identifier in the registry's AS, and the `iss` claim will identify the registry's AS.

This enables a registrar to implement fine-grained access control for its employees by assigning different scopes to different employee accounts in the registry's AS. For example, a junior employee may be granted only `domain:read` scope, while a senior employee may be granted `domain:create` and `domain:update` scopes. Support for this flow is OPTIONAL; a registrar MAY use the M2M flow for all operations if individual employee accountability is not required.

In this flow, the employee authenticates with the registry's AS. The registry acts as both the AS and the RPP resource server.

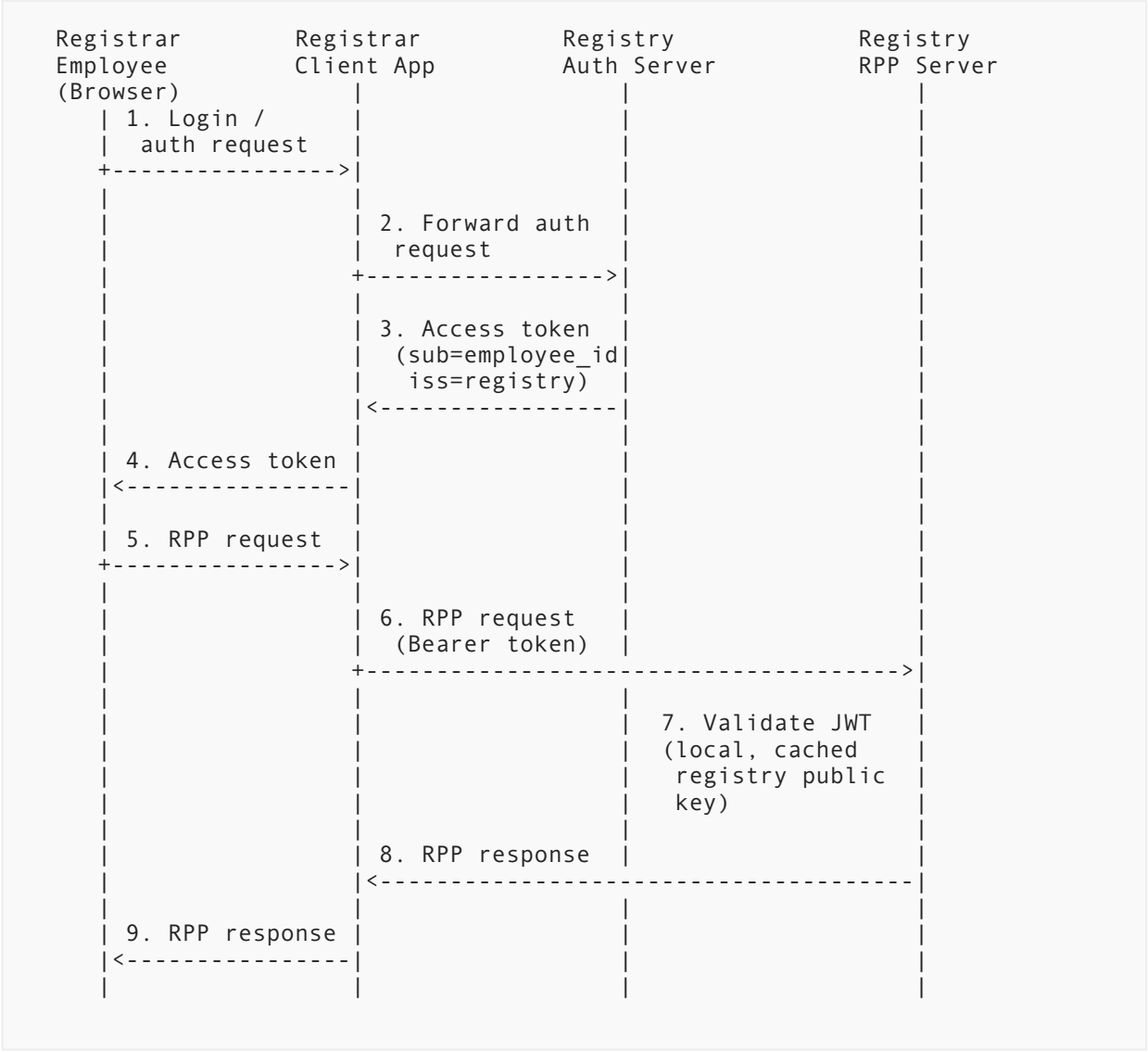

Figure 3: Interactive Flow - Registrar Employee

The steps in the diagram are as follows:

1. The registrar employee initiates a login or authorization request using the registrar's client application.
2. The client application forwards the authorization request to the registry's AS using the OAuth 2.0 Authorization Code grant.
3. The registry's AS authenticates the employee and issues a signed, short-lived JWT access token containing the granted scopes, `sub` (set to the employee's account identifier), and `rpp_registrar_id`. The `iss` claim identifies the registry's AS.
4. The client application returns the access token to the registrar employee.
5. The registrar employee submits an RPP request via the client application.

6. The client application forwards the RPP request to the registry's RPP server, including the access token in the HTTP `Authorization` header as a Bearer token.
7. The RPP server validates the JWT locally using the cached registry public key (fetched via OAuth 2.0 AS Metadata [RFC8414] and the referenced JWKS [RFC7517] endpoint). It verifies the signature, checks the standard claims (`iss`, `aud`, `exp`), and confirms the `scope` claim includes the scope required for the requested operation.
8. The RPP server processes the request and returns the RPP response to the client application.
9. The client application returns the RPP response to the registrar employee.

11.2.1. Example

Step 2 — Authorization Request (browser redirect to Registry AS):

```
GET /authorize
    ?response_type=code
    &client_id=registrar-app-client
    &redirect_uri=https%3A%2F%2Fclient.registrar.example%2Fcallback
    &scope=domain%3Acreate%20domain%3Aupdate
    &state=af0ifjsldkj
    &code_challenge=E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM
    &code_challenge_method=S256
Host: as.registry.example
```

Step 2 — Authorization Response (redirect back to client):

```
HTTP/1.1 302 Found
Location: https://client.registrar.example/callback
          ?code=SplxlOBeZQQYbYS6WxSbIA
          &state=af0ifjsldkj
```

Token Request (client exchanges authorization code for access token):

```
POST /token HTTP/1.1
Host: as.registry.example
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code
&code=SplxlOBeZQQYbYS6WxSbIA
&redirect_uri=https%3A%2F%2Fclient.registrar.example%2Fcallback
&client_id=registrar-app-client
&code_verifier=dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk
```

Token Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "scope": "domain:create domain:update"
}
```

The decoded JWT access token payload will contain claims similar to:

```
{
  "iss": "https://as.registry.example",
  "sub": "employee-42@registrar.example",
  "aud": "https://rpp.registry.example",
  "exp": 1746134400,
  "iat": 1746130800,
  "scope": "domain:create domain:update",
  "rpp_registrar_id": "REGISTRAR-001"
}
```

Step 6 — RPP Request (client sends RPP request with Bearer token):

```
GET /rpp/v1/domains/foo.example HTTP/1.1
Host: rpp.registry.example
Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json
```

RPP Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "name": "foo.example",
  ...
}
```

12. IANA Considerations

TODO

13. Internationalization Considerations

TODO

14. Security Considerations

TODO

15. Privacy Considerations

TODO

16. Change History

TODO

16.1. Version 00

- Created initial draft with core concepts and flows.

17. Normative References

[I-D.ietf-rpp-core] Wullink, M. and P. Kowalik, "RESTful Provisioning Protocol (RPP)", Work in Progress, Internet-Draft, draft-ietf-rpp-core-00, 6 July 2026, <https://datatracker.ietf.org/doc/html/draft-ietf-rpp-core-00>.

[I-D.ietf-rpp-data-objects] Kowalik, P. and M. Wullink, "RESTful Provisioning Protocol (RPP) Data Objects", Work in Progress, Internet-Draft, draft-ietf-rpp-data-objects-01, 3 July 2026, <https://datatracker.ietf.org/doc/html/draft-ietf-rpp-data-objects-01>.

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <https://www.rfc-editor.org/info/rfc2119>.

[RFC3986] Berners-Lee, T., Fielding, R., and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax", STD 66, RFC 3986, DOI 10.17487/RFC3986, January 2005, <https://www.rfc-editor.org/info/rfc3986>.

[RFC6749] Hardt, D., Ed., "The OAuth 2.0 Authorization Framework", RFC 6749, DOI 10.17487/RFC6749, October 2012, <https://www.rfc-editor.org/info/rfc6749>.

[RFC6750] Jones, M. and D. Hardt, "The OAuth 2.0 Authorization Framework: Bearer Token Usage", RFC 6750, DOI 10.17487/RFC6750, October 2012, <https://www.rfc-editor.org/info/rfc6750>.

[RFC7517] Jones, M., "JSON Web Key (JWK)", RFC 7517, DOI 10.17487/RFC7517, May 2015, <https://www.rfc-editor.org/info/rfc7517>.

[RFC7519] Jones, M., Bradley, J., and N. Sakimura, "JSON Web Token (JWT)", RFC 7519, DOI 10.17487/RFC7519, May 2015, <https://www.rfc-editor.org/info/rfc7519>.

[RFC7523] Jones, M., Campbell, B., and C. Mortimore, "JSON Web Token (JWT) Profile for OAuth 2.0 Client Authentication and Authorization Grants", RFC 7523, DOI 10.17487/RFC7523, May 2015, <https://www.rfc-editor.org/info/rfc7523>.

[RFC7617] Reschke, J., "The 'Basic' HTTP Authentication Scheme", RFC 7617, DOI 10.17487/RFC7617, September 2015, <https://www.rfc-editor.org/info/rfc7617>.

[RFC7636] Sakimura, N., Ed., Bradley, J., and N. Agarwal, "Proof Key for Code Exchange by OAuth Public Clients", RFC 7636, DOI 10.17487/RFC7636, September 2015, <https://www.rfc-editor.org/info/rfc7636>.

[RFC8414] Jones, M., Sakimura, N., and J. Bradley, "OAuth 2.0 Authorization Server Metadata", RFC 8414, DOI 10.17487/RFC8414, June 2018, <https://www.rfc-editor.org/info/rfc8414>.

[RFC8725] Sheffer, Y., Hardt, D., and M. Jones, "JSON Web Token Best Current Practices", BCP 225, RFC 8725, DOI 10.17487/RFC8725, February 2020, <https://www.rfc-editor.org/info/rfc8725>.

[RFC9068] Bertocci, V., "JSON Web Token (JWT) Profile for OAuth 2.0 Access Tokens", RFC 9068, DOI 10.17487/RFC9068, October 2021, <https://www.rfc-editor.org/info/rfc9068>.

[RFC9396] Lodderstedt, T., Richer, J., and B. Campbell, "OAuth 2.0 Rich Authorization Requests", RFC 9396, DOI 10.17487/RFC9396, May 2023, <https://www.rfc-editor.org/info/rfc9396>.

Acknowledgements

TODO

Authors' Addresses

Maarten Wullink
SIDN Labs
Email: maarten.wullink@sidn.nl
URI: https://sidn.nl/

Pawel Kowalik
DENIC
Email: pawel.kowalik@denic.de
URI: https://denic.de/